

1. ATM System (Custom Checked Exception)

```
import java.util.*;

// Custom Checked Exception
class InsufficientFundsException extends Exception {
    private double balance;
    private double amount;

    // Default Constructor
    public InsufficientFundsException() {
        balance = 0;
        amount = 0;
    }

    // Parameterized Constructor
    public InsufficientFundsException(double balance, double amount) {
        this.balance = balance;
        this.amount = amount;
    }

    // toString method
    public String toString() {
        return "Cannot withdraw " + amount + ". Available balance: " + balance;
    }
}

// ATM Class
class ATM {
    private double balance;

    // Default Constructor
    public ATM() {
        balance = 0;
    }

    // Parameterized Constructor
    public ATM(double balance) {
        this.balance = balance;
    }

    // Withdraw Method
    public void withdraw(double amount) throws InsufficientFundsException {

        if (amount <= 0) {
            throw new IllegalArgumentException("Withdrawal amount must be positive");
        }

        if (amount > balance) {
            throw new InsufficientFundsException(balance, amount);
        }

        balance -= amount;
        System.out.println("Withdrawal successful! " + amount + " deducted.");
    }
}
```

```

public double getBalance() {
    return balance;
}

// toString method
public String toString() {
    return "Current Balance: " + balance;
}
}

// Main Class
public class ATMTTest {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter initial balance: ");
        double bal = sc.nextDouble();

        ATM atm = new ATM(bal);

        System.out.print("Enter amount to withdraw: ");
        double amt = sc.nextDouble();

        try {
            atm.withdraw(amt);
        }
        catch (InsufficientFundsException e) {
            System.out.println("Checked Exception Caught:");
            System.out.println(e);
        }
        catch (IllegalArgumentException e) {
            System.out.println("Unchecked Exception Caught:");
            System.out.println(e.getMessage());
        }
        finally {
            System.out.println("\n--- Transaction Complete ---");
            System.out.println(atm);
        }

        sc.close();
    }
}

```

```

[annkurrerr@archlinux Q1]$ java ATMTTest
Enter initial balance: 5000
Enter amount to withdraw: 1500
Withdrawal successful! 1500.0 deducted.

--- Transaction Complete ---
Current Balance: 3500.0

```

2. Student Marks System (Multiple Exception Types)

```
import java.util.*;
```

```
// Custom Checked Exception
```

```
class InvalidMarksException extends Exception {  
    private int marks;
```

```
    // Default Constructor
```

```
    public InvalidMarksException() {  
        marks = 0;  
    }
```

```
    // Parameterized Constructor
```

```
    public InvalidMarksException(int marks) {  
        this.marks = marks;  
    }
```

```
    // toString method
```

```
    public String toString() {  
        return "Invalid Marks: " + marks + ". Marks should be between 0 and 100.";  
    }
```

```
}
```

```
// Student Class
```

```
class Student {  
    private String name;  
    private int marks;
```

```
    // Default Constructor
```

```
    public Student() {  
        name = "Unknown";  
        marks = 0;  
    }
```

```
    // Parameterized Constructor
```

```
    public Student(String name, int marks) {  
        this.name = name;  
        this.marks = marks;  
    }
```

```
    // Grade Calculation Method
```

```
    public String calculateGrade() {  
        if (marks >= 90) return "A";  
        else if (marks >= 75) return "B";  
        else if (marks >= 60) return "C";  
        else if (marks >= 40) return "D";  
        else return "F";  
    }
```

```
    // toString method
```

```
    public String toString() {  
        return "Student Name: " + name +  
            "\nMarks: " + marks +
```

```

        "\nGrade: " + calculateGrade();
    }
}

// Main Class
public class StudentMarksSystem {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {
            // Input Name
            System.out.print("Enter student name: ");
            String name = sc.nextLine();

            if (name == null || name.trim().isEmpty()) {
                throw new IllegalArgumentException("Student name cannot be empty");
            }

            // Input Marks
            System.out.print("Enter marks (0-100): ");
            int marks = sc.nextInt();

            if (marks < 0 || marks > 100) {
                throw new InvalidMarksException(marks);
            }

            // Create Object
            Student s = new Student(name, marks);

            System.out.println("\n--- Student Details ---");
            System.out.println(s);
        }

        // Catch Blocks
        catch (InvalidMarksException e) {
            System.out.println("Checked Exception Caught:");
            System.out.println(e);
        }
        catch (InputMismatchException e) {
            System.out.println("Input Error: Please enter integer value for marks.");
        }
        catch (IllegalArgumentException e) {
            System.out.println("Unchecked Exception Caught:");
            System.out.println(e.getMessage());
        }

        // Finally Block
        finally {
            System.out.println("\n--- Processing Completed ---");
            sc.close();
        }
    }
}

```

```
[annkurrrrr@archlinux Q2]$ java StudentMarksSystem
Enter student name: Ankur Kunde
Enter marks (0-100): 100

--- Student Details ---
Student Name: Ankur Kunde
Marks: 100
Grade: A

--- Processing Completed ---
```